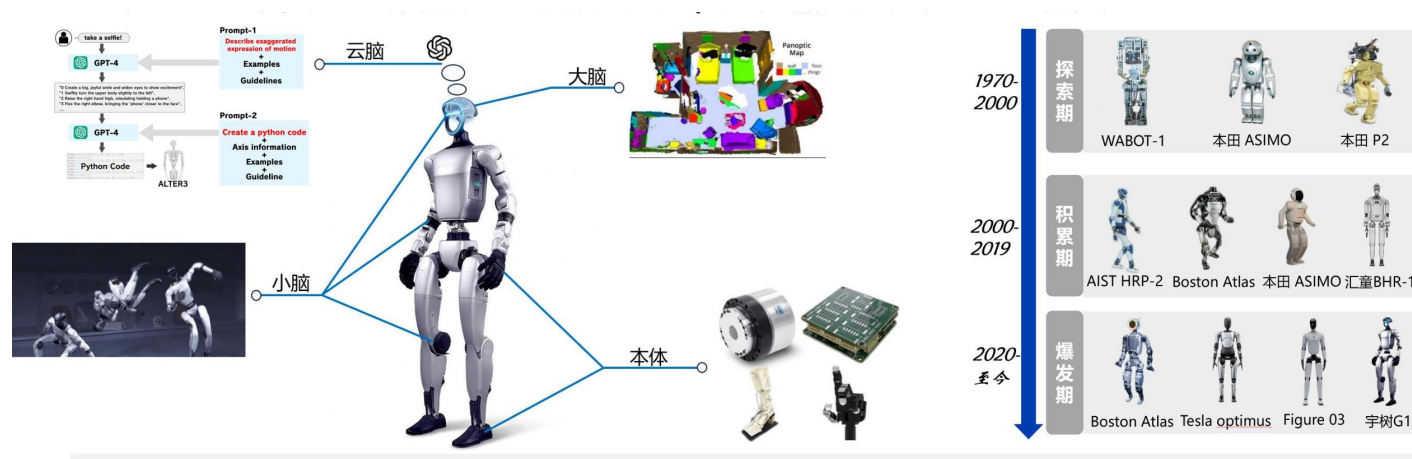


Part 2

0 Foreword



This part systematically introduces the basic knowledge and core principles of the Vision-Language-Action Model (VLA), reinforcement learning (RL), and imitation learning in humanoid robots. Imitation learning provides a fast path for robots to acquire basic behaviors by learning from human demonstrations; reinforcement learning, centered on the PPO algorithm, enables robots to optimize behaviors through environmental interaction, with its theoretical system starting from Markov Decision Process, passing through Bellman Equation and Policy Gradient Method, and finally converging to PPO. The VLA model, taking OpenVLA as the core, realizes the integration of "perception-understanding-execution" and has been iteratively improved through methods such as flow matching, $\pi_{\theta_{\text{vis}}}$, and dual-system architecture. The part elaborates on the core principles of the three technologies, combines practical operation cases of reinforcement learning, and summarizes key mathematical formulas, providing college students with a systematic theoretical foundation and practical reference oriented to humanoid robot applications.

1. Introduction

As a frontier intersection of artificial intelligence and robotics technology, humanoid robots are undergoing a major transformation from pre-programmed execution to intelligent autonomous decision-making. This transformation is jointly driven by three core technologies: reinforcement learning (RL), imitation learning, and the Vision-Language-Action (VLA) model. Among them, reinforcement learning, rooted in the Markov Decision Process, provides robots with powerful autonomous learning capabilities through continuous interaction with the environment, enabling them to optimize behavioral strategies via a trial-and-error mechanism

—an indispensable technology for high-dynamic motion control, as exemplified by Boston Dynamics’ Atlas and Unitree robots mastering balance control and complex movements. Imitation learning, as a complementary technology, allows robots to quickly acquire basic action skills by learning from human demonstrations, reducing the difficulty of initial training and laying a solid foundation for subsequent reinforcement learning optimization. Together, these two technologies address the basic action learning needs of humanoid robots, while the VLA model further breaks the bottleneck of "perception-understanding-execution" separation.

However, relying solely on reinforcement learning and imitation learning still faces limitations in human-centric scenarios: traditional robot control systems adopt a modular serial architecture, which is prone to error accumulation and poor generalization. Meanwhile, traditional Vision-Language Models (VLMs) can bridge the gap between perception and language but fail to generate executable actions, resulting in a disconnect between "knowledge" and "action." To solve this problem, the Vision-Language-Action (VLA) model emerges as an end-to-end multimodal framework, unifying visual perception, language understanding, and action generation into a single system. Taking OpenVLA as the core, the VLA model has undergone continuous iterative improvements (such as RT-2, π_0 , and Helix architectures), further enhancing the adaptability and execution efficiency of humanoid robots. This part systematically elaborates on the basic principles, core algorithms, and practical applications of imitation learning, reinforcement learning (focusing on PPO), and VLA models, providing a comprehensive theoretical and practical reference for the intelligent control of humanoid robots.

2. Basic Knowledge of Imitation Learning

Imitation learning (IL) is a key auxiliary technology for humanoid robot behavior learning, which enables robots to acquire action skills by imitating human demonstrations, avoiding the inefficiency of pure reinforcement learning in the initial training stage. Its core idea is to learn a policy that can replicate expert (human) behaviors from an expert demonstration dataset, which is particularly suitable for the initial training of humanoid robots for relatively complex tasks such as dancing, martial arts, and other similar movements.

The most widely used method in imitation learning for humanoid robots is Behavior Cloning (BC), which directly maps the observed state (e.g., visual input, joint angles) to the expert’s action through a neural network. The training objective of BC is to minimize the loss between the robot’s predicted action and the expert’s demonstration action, and its mathematical expression is:

$$L_{BC} = \mathbb{E}_{(s,a) \sim \mathcal{D}} \|\pi_{\theta}(s) - a_{\text{expert}}\|^2$$

Where:

- $\mathcal{D}$: Expert demonstration dataset, including the state s (joint angles,) and the corresponding expert action a_{expert} (joint torque, end-effector position);

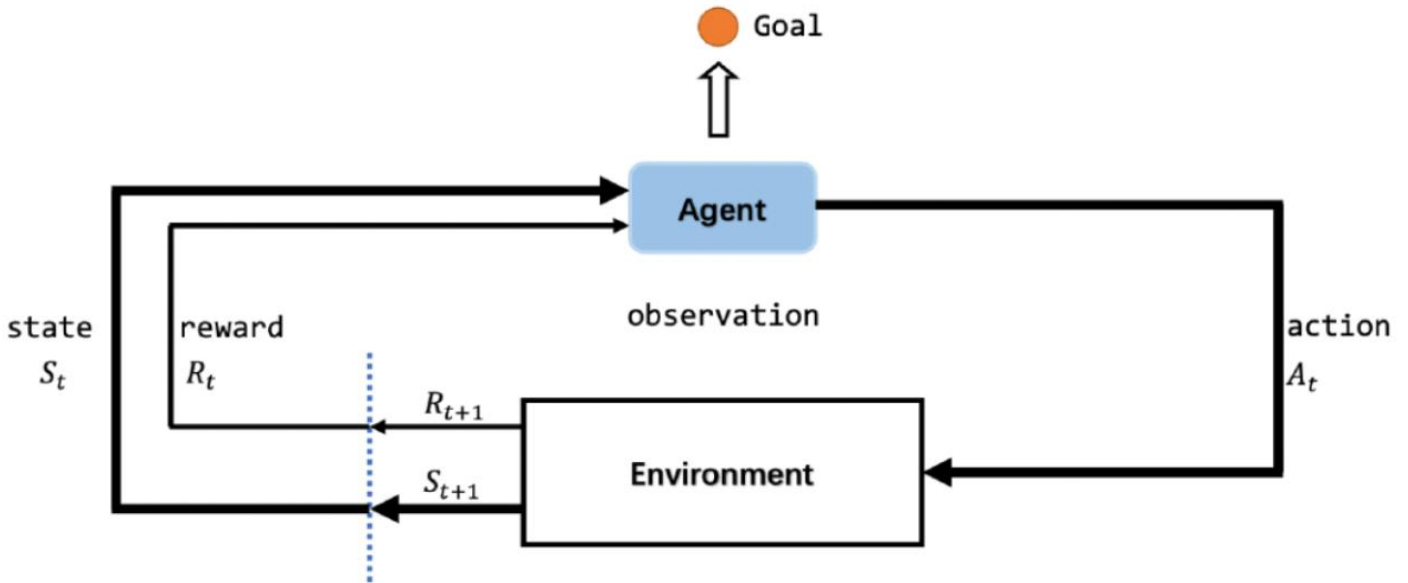
- $\pi_{\theta}(s)$: Deterministic policy network to be trained, parameterized by θ , outputting the robot' s predicted action vector under state s ;
- $|\cdot|^2$: L2 norm, measuring the difference between the predicted action and the expert action.

In humanoid robot applications, imitation learning and reinforcement learning are often combined via the teacher-student learning strategy, and DAGGER (Dataset Aggregation), a classic method for imitation learning, is commonly used to optimize the effect of this combination framework. Specifically, a "teacher model" trained by behavior cloning first helps the robot acquire basic actions from human demonstrations, solving RL' s initial convergence difficulty. Then, the "student model" (RL policy) is trained in real environments, with the teacher model providing guidance for policy optimization. DAGGER continuously aggregates the student' s new trajectory data, and the expert labels the optimal actions for each state in the trajectory to expand the expert demonstration dataset, which is then used to retrain the student model and reduce the distribution shift between demonstration and real data. This framework shortens the training cycle and ensures the safety and effectiveness of the robot' s initial behaviors while enhancing action robustness

3. Basic Knowledge of Reinforcement Learning (Focus on PPO)

3.1.1 Markov Decision Process (MDP): Mathematical Foundation of Robot-Environment Interaction

For humanoid robots, all dynamic control tasks can be abstracted as a **continuous-state, continuous-action Markov Decision Process (MDP)**, which is fundamentally different from discrete MDP for scenarios like chess games. The core of MDP is the **Markov Property**: given the current state of the robot, the future state only depends on the current state and executed action, and is independent of all historical states. This property greatly simplifies the modeling difficulty of humanoid robot control, as we only need to make action decisions based on the current real-time state.



The continuous MDP for humanoid robots consists of 5 core elements:

1. **State Space $\mathcal{S}$** : The set of all possible states of the humanoid robot, including continuous physical quantities such as whole-body joint angles, root position, root orientation, joint velocity, and center of mass position.
1. **Action Space $\mathcal{A}$** : The set of all executable actions of the robot, mainly continuous control commands such as joint torque, end-effector position, and movement speed (e.g., the torque range of each leg joint of the robot is $[-10, 10]\text{N} \cdot \text{m}$).
3. **Transition Probability $P(s'|s, a)$** : The probability that the robot transitions from the current state s to the next state s' after executing action a (e.g., the probability of maintaining balance after adjusting the knee joint torque).
4. **Reward Function $R(s, a, s')$** : The quantitative evaluation of the action a taken by the robot in state s , which is the core guidance for the robot to learn target behaviors (e.g., positive reward for stable standing and forward walking, negative reward for falling and joint over-limit).
5. **Discount Factor $\gamma \in [0, 1]$** : The weight of future rewards, which balances the robot's attention to immediate rewards and long-term task goals (e.g., completing a full walking cycle to obtain a higher cumulative reward).
6. **Policy π** : A core concept of RL, a mapping from state $s \in \mathcal{S}$ to action $a \in \mathcal{A}$, determining the robot's action in a given state. It is divided into two types: deterministic policy $a = \pi(s)$ (fixed action for a state) and stochastic policy $\pi(a|s)$ (probability distribution of actions, adopted for humanoid robots to enhance environmental exploration).
7. **Optimal Policy π^*** : The optimal mapping among all feasible policies, which maximizes the robot's expected cumulative discounted reward during interaction with the environment.

8. **Trajectory and Cumulative Return:** $\tau = (s_0, a_0, R_1, s_1, \dots, s_T)$ is the interaction trajectory of the robot and the environment; $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$ is the cumulative discounted return starting from time step t , a key indicator for policy optimization.

The interaction cycle between the robot (agent) and the environment is:

Observe state $S_t \rightarrow$ Select action A_t by policy $\rightarrow$ Transition to $S_{t+1} \rightarrow$
Obtain reward R_{t+1}

The ultimate goal of RL is to find the **optimal policy** π^* that maximizes the robot's expected cumulative discounted reward.

3.1.2 Bellman Equation: Quantifying State and Action Value

Based on MDP, the Bellman Equation quantifies the "goodness" of the robot's state and action, and provides the theoretical basis for policy evaluation and optimization. We first define two core value functions:

1. **State Value Function** $V^\pi(s)$: The expected cumulative reward obtained by the robot following policy π starting from state s . Its recursive Bellman Expectation Equation is:

$$V^\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$$

It describes the overall value of the current state, regardless of the specific action taken (e.g., the expected cumulative reward of the robot in a stable standing state).

2. **Action Value Function** $Q^\pi(s, a)$ (**Q-Function**): The expected cumulative reward obtained by the robot taking action a in state s and then following policy π . Its recursive Bellman Expectation Equation is:

$$Q^\pi(s, a) = \mathbb{E}_\pi [R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

It links the state and action, quantifying the value of a specific action in a specific state (e.g., the expected cumulative reward of adjusting the knee joint torque by a specific value in the standing state).

The state value function and action value function can be converted to each other through the following core equations, which further reflect their recursive relationship:

$$V^\pi(s) = \mathbb{E}_{\pi(a|s)} [Q^\pi(s, a) \mid S_t = s]$$

$$Q^\pi(s, a) = \mathbb{E}_{P(s'|s, a)} [R(s, a, s') + \gamma V^\pi(s') \mid S_t = s, A_t = a]$$

The first equation shows that the state value is the expected value of the action value under the current policy; the second equation combines the transition probability and discount factor, linking the action value to the immediate reward and the next state value.

The Bellman Equation reveals the core recursive relationship: the value of the current state depends on the immediate reward and the value of the next state. Based on this, there are two mainstream routes to find the optimal policy:

- **Value-Based RL:** First approximate the optimal action value function $Q^*(s, a)$, then derive the optimal policy from Q^* ;
- **Policy-Based RL:** Directly parameterize and optimize the policy π , without explicitly solving the value function first.

3.1.3 DQN: Value-Based Deep RL

The core of Value-Based RL is to approximate the optimal action value function $Q^*(s, a)$ with a neural network, which is represented by DQN (Deep Q-Network).

Core Principle

We use a neural network $Q(s, a; w)$ with parameter w to approximate the optimal $Q^*(s, a)$, and combine **Temporal Difference (TD) Learning** to update the parameters:

1. **TD Target:** $y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; w^-)$, where w^- is the parameter of the target network (fixed for a certain period to improve training stability);
2. **Loss Function:** Minimize the mean square error of the TD error:

$$L(w) = \mathbb{E} \left[(Q(s_t, a_t; w) - y_t)^2 \right]$$

3. **Optimization:** Update parameter w via gradient descent.

Core Limitation (Critical for Humanoid Robots)

DQN relies on the $\max_{a'} Q(s, a)$ operation to select the optimal action, which can only be solved analytically in discrete action space. For humanoid robots with high-dimensional continuous action space, this max operation has no analytical solution, and numerical solution will bring unbearable computational cost. This makes DQN completely unsuitable for humanoid robot control, so we turn to the Policy-Based RL route.

3.1.4 Policy Gradient Method: Direct Policy Optimization for Continuous Action Space

Policy-Based RL directly parameterizes the policy as $\pi_\theta(a|s)$ (a neural network with parameter θ), and optimizes θ by maximizing the expected cumulative reward, which perfectly adapts to the continuous action space of humanoid robots.

Core Derivation

Define the optimization objective (expected cumulative reward):

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right]$$

where $\tau = (s_0, a_0, R_1, s_1, \dots, s_T)$ is the interaction trajectory of the robot and the

environment, and the trajectory distribution is $P(\tau|\theta) = P(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$.

We calculate the gradient of $J(\theta)$ with respect to θ via the **score function trick** ($\nabla_{\theta} P(\tau|\theta) = P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta)$), and the derivation is as follows:

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int P(\tau|\theta) \cdot G_0 d\tau \quad (G_0 = \sum_{t=0}^{T-1} \gamma^t R_{t+1}, \text{total trajectory return}) \\
&= \int P(\tau|\theta) \cdot \nabla_{\theta} \log P(\tau|\theta) \cdot G_0 d\tau \\
&= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right) \cdot G_0 \right] \quad (\text{non-}\theta \text{ terms have 0 gradient}) \\
&= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t \right] \quad (\text{reward rearrangement, } G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1})
\end{aligned}$$

We finally get the **Policy Gradient Theorem**, the core formula for policy optimization:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t \right]$$

We update the policy parameter θ via gradient ascent to maximize $J(\theta)$.

Core Limitations

The traditional policy gradient method has three fatal defects for humanoid robot training:

1. **High Variance:** The Monte Carlo full-trajectory return G_t is affected by sampling noise, leading to extremely unstable gradient estimation, which is easy to cause joint control oscillation and even robot falling;
2. **Uncontrolled Update Step:** There is no constraint on the policy update amplitude, and excessive updates will lead to policy collapse (the robot suddenly outputs abnormal actions);
3. **Low Sample Efficiency:** It is an On-Policy method, and the sampled trajectory data can only be used once, which requires a huge amount of interaction data for high-dimensional humanoid robot training.

3.1.5 Algorithm Evolution: From Policy Gradient to PPO

We solve the above defects step by step, and finally derive the PPO algorithm, with a clear progressive logic:

Step 1: Actor-Critic Framework - Reduce Gradient Variance

To solve the high variance problem, we introduce the Actor-Critic framework, which combines policy optimization and value function approximation:

- **Actor (Policy Network):** Parameter θ , outputs the policy $\pi_{\theta}(a|s)$, updates via gradient ascent to maximize the cumulative reward;

- **Critic (Value Network):** Parameter w , outputs the state value function $V(s; w)$, updates via gradient descent to minimize the TD error, and is used to estimate the return.

We replace the high-variance Monte Carlo return G_t with the Q-value estimated by the Critic, and further introduce the **Advantage Function** to get a lower-variance policy gradient:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

The advantage function measures the superiority of action a_t compared to the average level of the policy in state s_t . The policy gradient is rewritten as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot A_t \right]$$

This framework greatly reduces the gradient variance, but still retains the defects of uncontrolled update step and low sample efficiency.

Step 2: Importance Sampling - Improve Sample Efficiency

To solve the On-Policy low sample efficiency problem, we introduce **Importance Sampling**, which enables us to reuse the data sampled by the old policy $\pi_{\theta_{\text{old}}}$ to update the new policy π_θ , converting it to an Off-Policy method.

Define the importance weight (probability ratio of new and old policies):

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

The optimization objective is rewritten as:

$$J(\theta) = \mathbb{E}_{(s,a) \sim \pi_{\theta_{\text{old}}}} [r_t(\theta) \cdot A_t]$$

This realizes the reuse of sampling data and greatly improves sample efficiency. However, when the difference between the new and old policies is too large, $r_t(\theta)$ will have extreme values, leading to gradient explosion and training instability.

Step 3: TRPO - Constrain Policy Update Amplitude

To solve the problem of excessive policy update, TRPO (Trust Region Policy Optimization) introduces a KL divergence constraint to limit the difference between the new and old policies, forming a constrained optimization problem:

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}_{(s,a) \sim \pi_{\theta_{\text{old}}}} [r_t(\theta) \cdot A_t] \\ \text{s.t.} \quad & \mathbb{E}_{s \sim \pi_{\theta_{\text{old}}}} [\text{KL}(\pi_{\theta_{\text{old}}}(\cdot | s) \parallel \pi_\theta(\cdot | s))] \leq \delta \end{aligned}$$

where $\text{KL}(p \parallel q)$ is the KL divergence, measuring the difference between two probability distributions, and δ is the trust region threshold.

TRPO ensures training stability, but it requires complex second-order optimization (conjugate gradient method), which has extremely high computational cost and is difficult to implement for large-scale training of high-dimensional humanoid robots.

Step 4: PPO - First-Order Simplification of TRPO

PPO (Proximal Policy Optimization) is a first-order simplified implementation of TRPO. It uses a simple clip operation to achieve the effect of trust region constraint, while retaining the advantages of TRPO's stability, and is easy to implement, efficient, and robust to continuous action space. This makes PPO the de facto standard algorithm for humanoid robot control.

3.1.6 PPO Algorithm: Core Implementation for Humanoid Robots

Core Objective Function (Clip Version, Industrial Standard)

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

Where:

- $r_t(\theta) = \frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)}$: Probability ratio of the new and old policies;
- A_t : Advantage function. In engineering implementation for humanoid robots, we use **GAE (Generalized Advantage Estimation)** to calculate A_t , which balances bias and variance via multi-step TD, making the advantage estimation more stable and avoiding joint control oscillation;
- ϵ : Clipping parameter, usually set to 0.2, which limits $r_t(\theta)$ to the interval $[1 - \epsilon, 1 + \epsilon]$, avoiding excessive policy updates.

The $\min$ operation ensures that:

- When $A_t > 0$ (the action is better than the average level), we limit the maximum update amplitude to avoid over-optimization;
- When $A_t < 0$ (the action is worse than the average level), we limit the wrong update amplitude to avoid policy collapse.

Full PPO Algorithm Flow for Humanoid Robots

1. **Sampling**: Interact with the environment using the current old policy $\pi_{\theta_{\text{old}}}$, collect trajectory data including state s_t , action a_t , reward r_t ;
2. **Advantage Estimation**: Calculate the advantage function A_t for each time step using GAE;
3. **Policy Update**: Maximize the clip objective function $L^{CLIP}(\theta)$, update the policy network (Actor) via gradient ascent;
4. **Value Network Update**: Minimize the value function loss $L^{VF}(\theta) = \mathbb{E}_t \left[(V(s_t; \theta) - R_t)^2 \right]$ (R_t is the cumulative discounted return), update the value network (Critic) via gradient descent;
5. **Iteration**: Repeat the above steps until the policy converges and the robot completes the target control task.

Key Advantages for Humanoid Robots

1. **Stability:** The clip mechanism strictly limits the policy update amplitude, avoiding policy collapse and robot falling caused by excessive updates;
2. **Adaptability:** Perfectly adapts to the high-dimensional continuous action space of humanoid robots;
3. **Sample Efficiency:** Supports multiple updates of a single batch of sampled data, greatly reducing the required interaction data;
4. **Simplicity:** Only requires first-order optimization, easy to implement and debug, suitable for large-scale engineering deployment.

4. Basic Knowledge of VLA Model (Focus on OpenVLA and Its Iteration)

4.1 Core Principle of VLA Model: Transformer-Based Multimodal Fusion

The Vision-Language-Action (VLA) model is an end-to-end multimodal framework that breaks the isolated "perception-understanding-execution" pipeline in traditional robot control systems. Its core design is to unify visual perception, natural language understanding, and robot action generation within a single Transformer architecture, enabling humanoid robots to flexibly execute natural language instructions and robustly adapt to unstructured dynamic environments. Unlike traditional modular systems where information is passed serially between independent perception, planning, and control modules, the VLA model achieves deep fusion of multimodal signals in a shared feature space, fundamentally avoiding error accumulation caused by cascaded module transmission.

The fundamental design philosophy of the VLA model is inspired by autoregressive Transformer models (e.g., GPT series). The core paradigm is to encode visual signals, language instructions, and robot proprioceptive states into discrete tokens, then fuse these heterogeneous tokens through a shared Transformer backbone, and finally generate executable robot control commands. The core mathematical framework of the general VLA paradigm is built on a unified token embedding space, and its overall workflow can be formalized by the following equations:

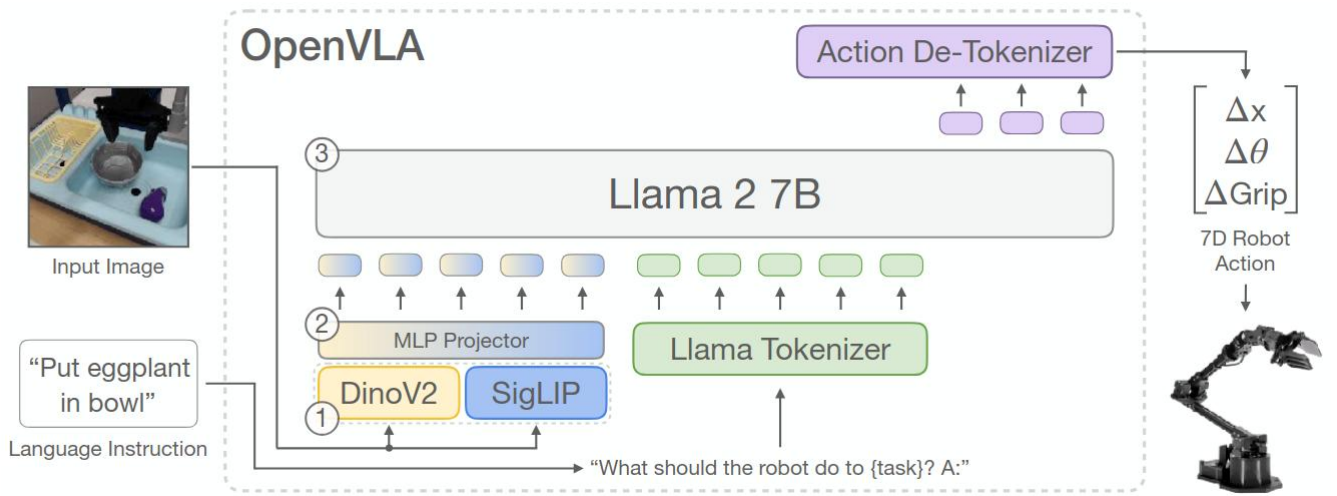
$$\mathcal{F}(v, l, s) = \text{MultiModalSelfAttention}(\text{Concat}(\text{ViT}(v), \text{LLM}(l), \text{MLP}(s)))$$

$$a_{1:T} = \text{TransformerDecoder}(\mathcal{F}(v, l, s))$$

Where the variables and operators are defined as follows:

- v : Visual input, typically RGB or RGB-D images of the operating environment captured by the humanoid robot's onboard camera;
- l : Natural language instruction that specifies the robot's task (e.g., "Grasp the red cup on the table and hand it to me");
- s : Robot proprioceptive state, including joint angles, end-effector 6D pose, and force-torque sensor data;
- $\text{ViT}(v)$: Vision Transformer encoder, which maps raw visual input into structured visual tokens;
- $\text{LLM}(l)$: Large Language Model encoder/embedding layer, which converts natural language instructions into semantic tokens with contextual information;
- $\text{MLP}(s)$: Multi-Layer Perceptron, which projects low-dimensional robot state signals into state tokens compatible with the Transformer backbone;
- $\text{Concat}(\cdot)$: Token concatenation operation, which aligns the sequence dimensions of visual, language, and state tokens to form a unified multimodal token sequence;
- $\text{MultiModalSelfAttention}(\cdot)$: Multi-modal self-attention mechanism, which aligns and fuses visual, language, and state tokens into a unified shared embedding space within a single attention computation;
- $a_{1:T}$: The generated T-step executable action sequence for robot control.

The Transformer architecture is the core enabler of the VLA model's powerful multimodal fusion and generalization capabilities. Unlike traditional modular systems that use independent models for each modality, the VLA model processes all modal tokens within a unified Transformer backbone, which guarantees end-to-end semantic, spatial, and temporal alignment across vision, language, and action. The self-attention mechanism of the Transformer allows the model to dynamically focus on task-critical information within each modality, for example, locating the target "red cup" in the visual input, extracting the core intent from the language instruction, and capturing the current motion limit of the robot arm from the proprioceptive state. Meanwhile, the cross-attention mechanism realizes bidirectional information interaction between different modalities, enabling the model to ground language semantics to visual spatial features, and map multimodal semantic understanding to robot action space—this is the core logic for the VLA model to achieve the "unity of perception, cognition and action".



As a representative state-of-the-art open-source VLA framework, OpenVLA further optimizes the Transformer-based multimodal fusion mechanism with targeted designs for real-world robot control scenarios. Built on the Prismatic Vision-Language Model (VLM) with a 32-layer Llama 2 7B decoder-only Transformer as its backbone, OpenVLA adopts a dual-branch visual encoder architecture to balance global semantic understanding and local manipulation precision: it uses DINOv2 to extract global semantic features of the scene, and SigLIP to capture fine-grained spatial features, with the two sets of visual features fused via channel concatenation and projected to the language embedding space through a lightweight MLP. Different from the general VLA paradigm, OpenVLA takes RGB images and natural language instructions as core inputs, and integrates robot proprioceptive state normalization into the action tokenization pipeline instead of using an independent MLP branch for state encoding.

For action generation, OpenVLA adopts a single-step action prediction paradigm rather than multi-step autoregressive sequence generation. To adapt continuous robot control signals to the discrete token space of the Transformer backbone, OpenVLA uniformly discretizes each dimension of the robot action space into 256 bins within the 1%-99% quantile range of the training dataset, and maps these bins to the least frequently used tokens in the Llama 2 vocabulary, avoiding complex frequency-domain transformation or token compression operations.

OpenVLA is pre-trained on 970,000 real-world robot demonstration trajectories from the Open X-Embodiment (OXE) and DROID datasets. To reduce the computational overhead of downstream adaptation while preserving the model's generalization capability, it supports efficient fine-tuning via Low-Rank Adaptation (LoRA), with verified optimal performance when tuning only the attention layers of the Transformer backbone with a rank of 32 to 64. The core advantage of OpenVLA is its strong cross-embodiment zero-shot transfer capability: it can directly transfer manipulation policies trained on source robot platforms to target humanoid robots with different joint configurations and kinematic structures, without additional full-

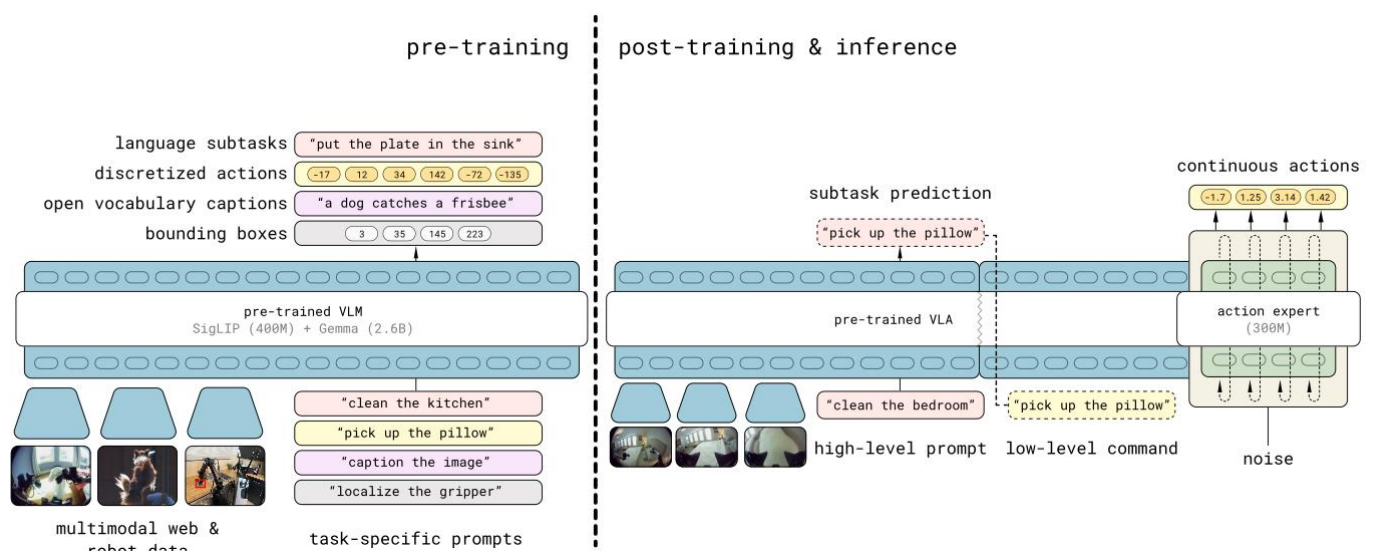
model fine-tuning, making it highly suitable for the research and development of general-purpose humanoid robots.

4.2 Iterative Improvements of VLA Model: Representative Architectures and Principle Innovations

Based on OpenVLA’s core framework, VLA model iterations focus on solving humanoid robots’ pain points of real-time control, computational efficiency and complex scene adaptation. $\pi_{0.5}$, the derivative iterative architecture of π_0 , introduces Flow Matching to fix core limitations of discrete token-based VLAs, optimized for humanoid continuous joint control. To address inherent bottlenecks of single/dual-system architectures, the triple-system architecture decouples semantic reasoning, whole-body action generation and bipedal dynamic balance control, natively adapting to humanoid loco-manipulation requirements and eliminating performance trade-offs of traditional designs.

4.2.1 $\pi_{0.5}$: Advanced VLA Architecture with Flow Matching Mechanism

$\pi_{0.5}$ is an advanced iterative Vision-Language-Action (VLA) architecture built on its predecessor π_0 . Its core innovation is integrating the Flow Matching mechanism to solve key limitations of discrete token-based VLA models (including OpenVLA): low autoregressive decoding efficiency, poor continuity of generated actions, and precision loss in continuous robot control. Its core design follows a hybrid paradigm: pre-training the model with discrete action tokens to retain high training efficiency and cross-modal alignment capability, then adopting Flow Matching for end-to-end continuous action generation via ODE solving, realizing efficient alignment between discrete pre-training and continuous execution — optimized for continuous joint control scenarios of humanoid robots.



Flow Matching is a continuous generative model, whose core workflow forms a closed loop of **ODE gradient field fitting in training phase** and **ODE initial value problem extrapolation**

solving in inference phase. During training, the model learns a continuous vector flow field, which essentially fits the gradient direction of the ordinary differential equation (ODE) that guides Gaussian random noise to evolve into real robot actions. During inference, the model maps random noise to executable continuous robot action sequences by solving the ODE via numerical forward extrapolation.

The core learning objective of Flow Matching is the flow function $\mathbf{f}_\theta(a, \tau, o, \ell)$, which gradually maps random noise $\omega \sim \mathcal{N}(0, I)$ to the ground-truth action sequence $a_{t:t+H}$ along the time index $\tau \in [0, 1]$, where o is robot observation, ℓ is the natural language instruction. The target vector field predicted by the network $\mathbf{f}_\theta$ is exactly the time derivative term of the ODE governing action state evolution. The engineering-optimized mathematical framework used in $\pi 0.5$ is as follows:

1. Noisy Action Sample Construction

Noisy samples for flow field learning are built by linear interpolation between ground-truth action sequences and random noise via the time index τ , which provides supervision signals for ODE gradient fitting:

$$a_{t:t+H}^{\tau, \omega} = \tau \cdot a_{t:t+H} + (1 - \tau) \cdot \omega$$

Where H is the action sequence length (fixed to 49~50 steps in $\pi 0.5$ to meet the continuity requirements of humanoid robot actions), $\tau = 0$ corresponds to pure noise (initial state of ODE solving), and $\tau = 1$ corresponds to the ground-truth action sequence (target state of ODE solving). To improve the stability of ODE fitting and convergence speed during inference, $\pi 0.5$ adopts non-uniform sampling of τ via Beta distribution during training, with an emphasis on low- τ samples, instead of uniform sampling over $\tau \in [0, 1]$.

2. Flow Matching Objective Function (ODE Gradient Fitting)

The core objective of model training is to make the learned flow field infinitely close to the theoretical optimal gradient direction of the ODE, and the fitting is completed by minimizing the flow field prediction error:

$$\mathcal{L}_{flow} = \mathbb{E}_{\mathcal{D}, \tau, \omega} \left| \mathbf{f}_\theta(a_{t:t+H}^{\tau, \omega}, o_t, \ell) - (a_{t:t+H} - \omega) \right|^2$$

Where $\mathcal{D}$ is the robot demonstration dataset, o_t is the robot observation (visual input + proprioceptive state) at t , and $a_{t:t+H} - \omega$ is the theoretical optimal gradient direction of the ODE.

3. ODE Inference & Forward Extrapolation for Action Generation

This is the core link of generating executable actions from the trained flow field, which is essentially **forward numerical extrapolation solving of the ODE initial value problem**.

After training, the learned flow function $\mathbf{f}_\theta$ is exactly the time derivative of the ODE that controls the evolution of the action state, with the core ODE equation:

$$\frac{da}{d\tau} = \mathbf{f}_\theta(a, \tau, o_t, \ell)$$

The inference process is to solve the initial value problem of this ODE: we take the random noise $a(0) = \omega \sim \mathcal{N}(0, I)$ with dimensions strictly matching the action space at $\tau = 0$ as the initial state, condition on the current visual observation, language instruction and proprioceptive state throughout the process, and extrapolate the action state step by step along the learned flow field via numerical method until $\tau = 1$, to obtain the final continuous action sequence.

To meet the real-time control requirements of humanoid robots, $\pi 0.5$ adopts the industry-standard forward Euler method to complete efficient extrapolation solving in only 10 steps, with the iteration formula:

$$a(\tau_{k+1}) = a(\tau_k) + (\tau_{k+1} - \tau_k) \cdot \mathbf{f}_\theta(a(\tau_k), \tau_k, o_t, \ell)$$

Where $\tau_k = k/10$ for $k \in [0, 9]$, and the final output $a(1)$ is the complete continuous action sequence that can be directly executed by the robot.

This fully non-autoregressive ODE extrapolation generation mode completely avoids the step-by-step decoding delay of discrete token models, which is the core of realizing low-latency and high-smoothness continuous action generation.

4. Joint Training Loss

$\pi 0.5$ combines discrete token pre-training and continuous flow field learning with a hybrid loss function to balance training efficiency and action quality:

$$\mathcal{L}_{\text{total}} = \mathbb{E}_{\mathcal{D}, \tau, \omega} [H(x_{1:M}, f_\theta^\ell(o_t, \ell)) + \alpha \cdot \mathcal{L}_{\text{flow}}]$$

Where $H(\cdot)$ is the cross-entropy loss for discrete token prediction (covering text, vision and discrete action token pre-training tasks), and α is the weight balancing the two loss terms, fixed to 10.0 in the post-training phase of $\pi 0.5$.

Architecture & Practical Advantage

$\pi 0.5$'s Transformer backbone is built on PaliGemma 2B (18-layer decoder-only architecture), with an added lightweight 5-layer *Action Expert* sub-network dedicated to flow field fitting and ODE extrapolation solving. This module unidirectionally attends to fused vision-language-state features from the main backbone, with strict attention masking to avoid information leakage between discrete token training and continuous ODE flow field learning. This design retains the high efficiency of discrete pre-training while enabling robust continuous action generation.

Compared with discrete-only VLA models like OpenVLA, this framework delivers significantly smoother continuous actions and lower inference latency via ODE-based generation, making it well-suited for real-time humanoid robot control scenarios including bipedal walking and bimanual manipulation.

4.2.2 Triple-System Architecture: VLA Paradigm for Humanoid Whole-Body Loco-Manipulation

1. Core Motivation

The core difference between humanoid robots and fixed-base manipulators lies in three inherently conflicting rigid requirements that humanoids must satisfy simultaneously:

1. Complex semantic reasoning for long-horizon tasks, which relies on the strong cognitive capability of large models and has high tolerance for inference latency;
2. Precise whole-body action generation for 30+ degrees of freedom, which demands high-frequency, smooth joint control and has strict requirements for model lightweighting and inference efficiency;
3. Real-time dynamic balance control for the bipedal floating base, which requires sub-millisecond closed-loop control, is extremely sensitive to latency, and any inference freeze in the upper layer will directly cause the robot to fall.

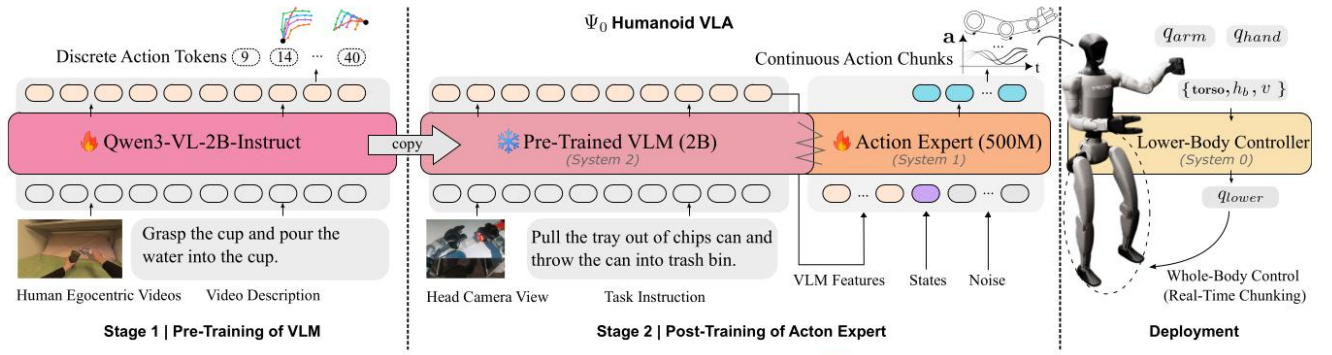
Traditional architectures are fundamentally unable to resolve this core contradiction:

- For single-system architectures (e.g., RT-2, OpenVLA, $\pi_{0.5}$): They couple all three functions into one monolithic model. To reduce latency for stable control, they have to sacrifice model capacity, which eventually leads to comprehensive degradation in both reasoning ability and action precision, creating an unsolvable dilemma between inference capability and control frequency.
- For dual-system architectures (e.g., Helix): They only split semantic reasoning and action generation, but still fail to isolate the unique real-time dynamic balance control exclusive to humanoid robots. Action generation and balance control remain coupled and conflicting, making it impossible to achieve stable execution of "walking while manipulating and reasoning" — the core scenario for humanoid robots.

To address these inherent bottlenecks, the triple-system architecture is proposed with a clear core design purpose: to completely decouple these three conflicting core requirements into three independent closed-loop yet deeply collaborative subsystems. Each subsystem only focuses on one core objective, with no need to make performance compromises between conflicting demands. This hierarchical design is natively aligned with the inherent control requirements of humanoid robots, and fundamentally eliminates the performance trade-offs of single and dual-system designs.

2. Representative Triple-System Designs

Ψ_0 (Psi-Zero) Triple-System Framework



As a state-of-the-art open foundation model for humanoid loco-manipulation, Ψ_0 adopts a standardized triple-system hierarchy:

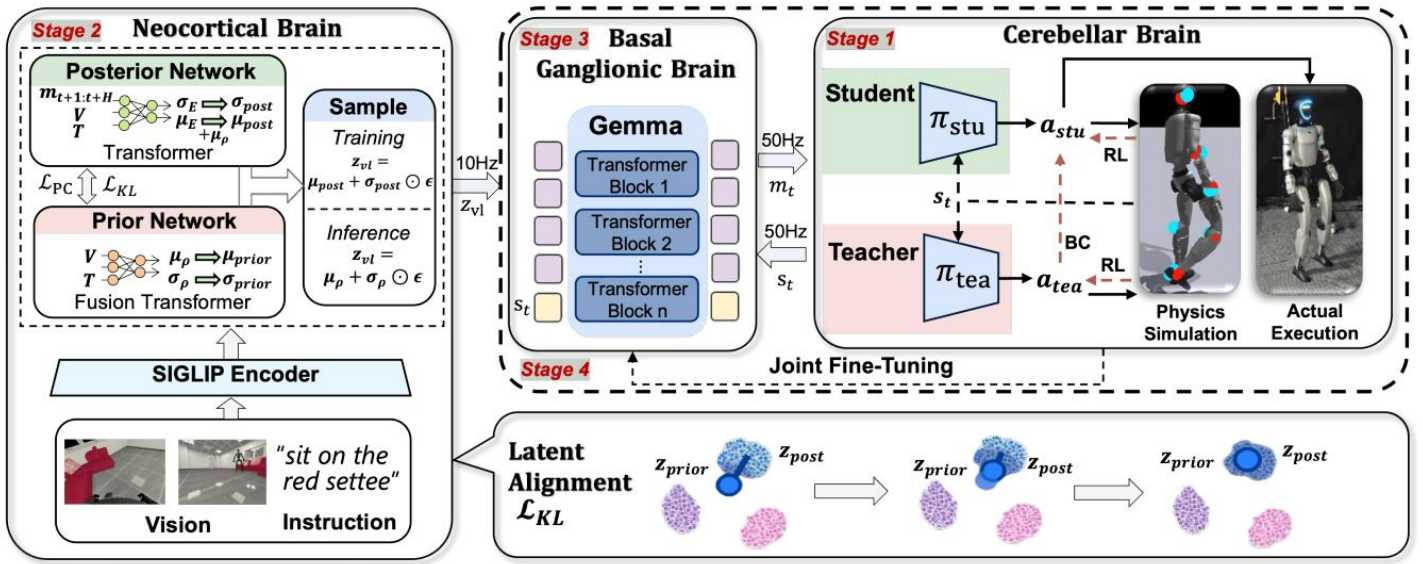
System 2 (Slow Thinking, Semantic Reasoning): Corresponding to the top-layer design objective, it adopts a Qwen3-VL-2B-Instruct vision-language backbone, pre-trained on large-scale egocentric human videos. It is dedicated to learning generalizable visual-action representations, task semantics, and long-horizon task planning capabilities, completely free from real-time control constraints.

System 1 (Fast Thinking, Action Generation): Corresponding to the middle-layer design objective, it is a 500M-parameter flow-based MM-DiT action expert. Conditioned on the frozen VLM features from System 2, it predicts precise whole-body joint-space action chunks, focusing on upper-body dexterous manipulation and coordinated torso control, with no need to handle balance control.

System 0 (Physical Execution, Real-Time Stabilization): Corresponding to the bottom-layer design objective, it is an RL-based lower-body controller (AMO). It maps high-level locomotion commands to 15-DoF lower-body joint angles, and is exclusively responsible for bipedal walking, dynamic balance, and torso stability in real time. This independent closed-loop is completely unaffected by upper-layer inference latency, serving as the core stability base of the robot.

PhysiFlow Bio-Inspired Multi-Brain Triple-System Design

Our proposed PhysiFlow framework further optimizes the triple-system architecture with a bio-inspired design for physics-aware humanoid control:



Neocortical Brain (Semantic-Motion Intent Alignment): Corresponding to the top-layer semantic reasoning objective, it is a curriculum-based CVAE with a SigLIP encoder. It fuses vision-language instructions to generate 10Hz semantic-motion latent vectors, and is responsible for complex task understanding and motion intent reasoning, isolated from real-time control constraints.

Basal Ganglionic Brain (High-Frequency Action Generation): Corresponding to the middle-layer action generation objective, it is a lightweight Gemma-based latent flow-matching model. Taking semantic latent vectors and robot states as input, it outputs 50Hz continuous whole-body motion sequences, balancing action smoothness and inference efficiency for whole-body coordinated control.

Cerebellar Brain (Physics-Aware Stabilization): Corresponding to the bottom-layer stability control objective, it is an RL teacher-student motion tracker with an internal 1000Hz PD controller. It enforces physical feasibility constraints and converts motion sequences into stable motor commands, ensuring dynamic balance and precise execution for humanoid whole-body control with an independent hard real-time closed loop.

(注：文档部分内容可能由 AI 生成)